Listing 4.37 Indexing a doctor document

```
PUT doctors/_doc/1
{
  "name":"Dr. Mary Montgomery",
  "relationship":{
    "name":"doctor"
  }
}
```

The relationship attribute must be one of the relations.

The `relationship` object declares that the type of the document is `doctor`. The `name` attribute must be the parent value (`doctor`) as declared in the mapping schema. Now that Dr. Mary Montgomery is ready, the next step is to associate two patients with her.

Listing 4.38 Creating two patients for our doctor

```
PUT doctors/_doc/2?routing=mary
{
  "name":"John Doe",
  "relationship":{
    "name":"patient",
    "parent":1
  }
}

PUT doctors/_doc/3?routing=mary
{
  "name":"Mrs. Doe",
  "relationship":{
    "name":"patient",
    "parent":1
  }
}
```

Documents must have the routing flag set.

We define the type of relationship in this object.

The document is a patient.

The patient's parent (doctor) is the document with ID 1.

The `relationship` object should have the value `patient` (remember the parent-child portion of the `relations` attribute in the schema?), and `parent` should be assigned a document identifier of the associated doctor (ID 1 in our example).

We need to understand one more thing when working with parent-child relationships. Parents and associated children are indexed into the *same shard* to avoid multi-shard search overheads. And as the documents coexist, we need to use a mandatory routing parameter in the URL. (Routing is a function that determines the shard where the document resides; we look at the routing algorithm in chapter 5.)

Finally, it's time to search for patients belonging to a doctor with ID 1. The following query searches for all the patients associated with Dr. Montgomery.

Listing 4.39 Fetching patients of Dr. Montgomery

```
GET doctors/_search
{
  "query": {
    "parent_id":{
```